

phaseGP API Reference

Version 0.2.0

Changes in 0.2.0

- Informed priors (`MultiGPMeanModule` , `SKMultiGPMeanModule` , and mean modules that wrap another `PhaseGP`) are now evaluated at the correct input locations when `min_scale / max_scale` are not 0/1 (inputs were previously scaled twice).
- `PhaseGP.fit` trains on the 0/1 labels directly; the previous logit transformation of the labels was removed (`epsilon` is kept only for backward compatibility).
- New `SharpBernoulliLikelihood` : labels are treated as certain measurements (`likelihood_sharpness=6.0` by default).
- `PhaseTransferGP` and `SKPhaseTransferGP` read the source weights with `weight_readout_sharpness=1.0` , the scale on which the weighting formulas were tuned.
- `SKMultiGPMeanModule` maps scikit-learn probabilities to the latent space with the probit (inverse normal CDF) instead of the logit.

Results obtained with 0.2.0 differ numerically from those obtained with earlier versions.

Navigation

[models.py](#)

[priors.py](#)

[source_pruner.py](#)

[utils.py](#)

[visualization.py](#)

models.py

Gaussian Process models for binary phase classification with active learning and transfer learning capabilities.

Classes

PhaseGP

Variational Gaussian Process model for binary phase classification.

```
PhaseGP(train_x, min_scale=None, max_scale=None, kernel_choice='matern32',
        lengthscale=0.3, learning_rate=0.1, training_iterations=120,
        lengthscale_interval=(0.2,0.3), outputscale_interval=(1.0,4.0),
        scale_inputs=True, device="cpu", likelihood_sharpness=6.0)
```

Parameter	Type	Default	Description
train_x	torch.Tensor	required	Initial training inputs used as inducing points, shape (n, d)
min_scale	array-like	None	Minimum values for input scaling (0 if None)
max_scale	array-like	None	Maximum values for input scaling (1 if None)
kernel_choice	str	'matern32'	Type of kernel: 'rbf', 'matern32', or 'matern52'
lengthscale	float	0.3	Initial kernel lengthscale (in scaled [0,1] units)
learning_rate	float	0.1	Optimization learning rate
training_iterations	int	120	Number of training iterations
lengthscale_interval	tuple	(0.2, 0.3)	Prior bounds for lengthscale parameter
outputscale_interval	tuple	(1.0, 4.0)	Prior bounds for outputscale parameter
scale_inputs	bool	True	Whether to internally scale the inputs to [0,1] using min_scale and max_scale
device	str	"cpu"	Model's device ("cpu" or "cuda")
likelihood_sharpness	float	6.0	Sharpness β of the probit link $p(y=1 f) = \Phi(\beta \cdot f)$. Values > 1 treat the labels as certain measurements; 1.0 is the standard Bernoulli likelihood

Methods:

- `fit(train_x, train_y, epsilon=0.05, verbose=False)` - Train the model on binary labels (0 or 1) using variational inference. `epsilon` is unused and kept for backward compatibility
- `predict(x)` - Predict binary phase labels
- `predict_proba(x)` - Predict the probability of phase 1
- `acquisition(x, sampled_points=None, epsilon=0.05, vanilla_acq=True, distance_acq=False, requires_grad=False)` - Acquisition values $\sigma(x) / (|\mu(x)| + \epsilon)$ for active learning. `sampled_points`, `vanilla_acq` and `distance_acq` are accepted for interface compatibility and are not used by this model
- `forward(x)` - Forward pass through the GP model. The covariance module receives scaled inputs; the mean module receives inputs in their original units

PhaseTransferGP

Transfer Learning GP model for phase classification with multiple source models. Each source model has a weight model (a `PhaseGP` trained on whether the source predicts the measured labels correctly) that determines how much the source is trusted at each location.

```
PhaseTransferGP(source_model_list, train_x, min_scale=None, max_scale=None,
                prior_aggregation="linear", max_adaptive_power=5,
```

```
explorative_threshold=0.4, kernel_choice='matern32',
lengthscale=0.3, learning_rate=0.1, training_iterations=120,
lengthscale_interval=(0.2,0.3), outputscale_interval=(1.0,4.0),
scale_inputs=True, device="cpu", likelihood_sharpness=6.0,
weight_readout_sharpness=1.0)
```

Additional parameters beyond PhaseGP:

Parameter	Type	Default	Description
source_model_list	list	required	List of pre-trained source models
prior_aggregation	str	'linear'	Method for aggregating priors: 'linear' or 'highest'
max_adaptive_power	float	5	Maximum power for adaptive weighting
explorative_threshold	float	0.4	Threshold for exploration vs exploitation (0 to 1)
likelihood_sharpness	float	6.0	Sharpness of the probit link used to train all internal PhaseGPs (weight and target models)
weight_readout_sharpness	float	1.0	Sharpness used to turn the weight models' latent function into source weights after training. The weight-dependent formulas (prior aggregation, adaptive power, explorative threshold) are tuned for 1.0

Methods:

- `fit(train_x, train_y, epsilon=0.05, verbose=False)` - Train the weight models and the target model
- `predict(x)` - Predict binary phase labels
- `predict_proba(x, return_all_data=False)` - Predict phase probabilities using the weighted combination of source and target; with `return_all_data=True` also returns (source probabilities, target probabilities, weights)
- `get_weight(x, requires_grad=False)` - Get the weight of the best source model at each input point
- `acquisition(x, sampled_points=None, epsilon=0.05, vanilla_acq=False, distance_acq=True, requires_grad=False)` - Compute acquisition function for transfer learning; `sampled_points` is required when `distance_acq=True`
- `forward(x, return_all_data=False)` - Forward pass combining source and target model predictions

SKPhaseTransferGP

Transfer Learning GP with scikit-learn compatible source models. Requires scikit-learn source models to have `predict()` and `predict_proba()` methods.

Inherits from PhaseTransferGP with the same parameters, except `scale_inputs` (always True). Source probabilities enter the prior through `SKMultiGPMeanModule`.

Methods:

- `chain_predict(x)` - Binary prediction using the minimum of the ensemble and source probabilities

SharpBernoulliLikelihood

Bernoulli likelihood with a sharpened probit link, $p(y=1 | f) = \Phi(\text{sharpness} \cdot f)$. A sharpness above 1 makes a misclassified label very unlikely for a given latent value, so the labels are treated as certain measurements without inflating the latent scale. `sharpness=1.0` is equivalent to gpytorch's `BernoulliLikelihood`.

```
SharpBernoulliLikelihood(sharpness=1.0)
```

Parameter	Type	Default	Description
sharpness	float	1.0	Fixed multiplier applied to the latent function in the probit link

Functions

train_gp_model

Train a GP model with variational inference (Adam optimizer, variational ELBO).

```
train_gp_model(model, train_x, train_y, learning_rate, training_iterations,  
               verbose=False, device="cpu", likelihood_sharpness=1.0)
```

Parameter	Type	Default	Description
model	ApproximateGP	required	GP model to train
train_x	torch.Tensor	required	Training inputs, shape (n, d)
train_y	torch.Tensor	required	Binary training labels (0 or 1), shape (n,)
learning_rate	float	required	Learning rate for the Adam optimizer
training_iterations	int	required	Number of optimization iterations
verbose	bool	False	Whether to print training progress
device	str	"cpu"	Device where the training is performed
likelihood_sharpness	float	1.0	Sharpness of the probit link (PhaseGP.fit passes the model's likelihood_sharpness)

Returns: tuple (model, likelihood) - Trained model and its SharpBernoulliLikelihood

priors.py

Informed prior mean functions for transfer learning. Source predictions in the latent space, weighted by their learned reliability, are used as the mean function of the target GP. Source and weight models are frozen (evaluation mode, no gradients).

Classes

MultiGPMeanModule

Prior mean from multiple GP source models. Used by `PhaseTransferGP`.

```
MultiGPMeanModule(source_model_list, weight_model_list, prior_aggregation="linear",
device="cpu")
```

Parameter	Type	Default	Description
<code>source_model_list</code>	list	required	Pre-trained PhaseGP source models
<code>weight_model_list</code>	list	required	PhaseGP weight models, one per source
<code>prior_aggregation</code>	str	'linear'	'linear': $\sum_i \mu_i(x) \cdot w_i(x)^{(\mu_i(x) +1)}$; 'highest': $w(x) \cdot \mu(x)$ of the most reliable source
<code>device</code>	str	"cpu"	Device where the prior is stored

SKMultiGPMeanModule

Prior mean from scikit-learn compatible source models. Used by `SKPhaseTransferGP`. Source probabilities p are mapped to the latent space with the probit transformation $f = \Phi^{-1}(\text{clamp}(p, \epsilon, 1-\epsilon))$, matching the probit likelihood of the target GP; the aggregation methods are the same as in `MultiGPMeanModule`.

```
SKMultiGPMeanModule(source_model_list, weight_model_list, prior_aggregation="linear",
epsilon=0.05, device="cpu")
```

Parameter	Type	Default	Description
<code>source_model_list</code>	list	required	scikit-learn compatible source models
<code>weight_model_list</code>	list	required	PhaseGP weight models, one per source
<code>prior_aggregation</code>	str	'linear'	Aggregation method: 'linear' or 'highest'

epsilon	float	0.05	Clamping value for the probit transformation (latent values within about ± 1.64)
device	str	"cpu"	Device where the prior is stored

GPMeanModule

Prior mean from a single GP source model: $\mu_{\text{source}}(x) \cdot w(x)$. Not used by the transfer models; MultiGPMeanModule with one source behaves similarly.

```
GPMeanModule(source_model, weight_model)
```

source_pruner.py

Source model selection algorithms for identifying diverse source models in transfer learning.

Functions

source_model_pruner

Select diverse source models from a collection based on phase diagram similarity.

```
source_model_pruner(source_model_list, x_min=None, x_max=None,
                    y_min=None, y_max=None, grid_size=100,
                    acc_threshold=0.9, min_intersection_regions=3,
                    intersection_tol=5, return_index=False)
```

Parameter	Type	Default	Description
source_model_list	list	required	List of trained phase classification models
x_min	float	None	Minimum x-coordinate for evaluation grid
x_max	float	None	Maximum x-coordinate for evaluation grid
y_min	float	None	Minimum y-coordinate for evaluation grid
y_max	float	None	Maximum y-coordinate for evaluation grid
grid_size	int	100	Number of grid points in each dimension
acc_threshold	float	0.9	Similarity threshold for pixel-wise accuracy
min_intersection_regions	int	3	Maximum disagreement regions for similarity
intersection_tol	int	5	Boundary tolerance in pixels
return_index	bool	False	If True, also return indices of selected models

Returns: List of selected diverse models, or tuple (selected_models, selected_indices) if return_index=True

similarity_checker

Check if two phase diagrams are similar based on multiple criteria.

```
similarity_checker(phase_diagram1, phase_diagram2, acc_threshold=0.8,
                  min_intersection_regions=3, intersection_tol=5)
```

Parameter	Type	Default	Description
phase_diagram1	np.ndarray	required	First binary phase diagram
phase_diagram2	np.ndarray	required	Second binary phase diagram
acc_threshold	float	0.8	Minimum required pixel-wise accuracy (0-1)
min_intersection_regions	int	3	Maximum allowed disagreement regions
intersection_tol	int	5	Dilation radius for boundary tolerance (pixels)

Returns: bool - True if diagrams are similar, False otherwise

source_diagram_pruner

Select diverse phase diagrams from a collection based on similarity.

```
source_diagram_pruner(phase_diagram_list, acc_threshold=0.9,
                      min_intersection_regions=3, intersection_tol=5)
```

Parameter	Type	Default	Description
phase_diagram_list	list	required	List of 2D binary phase diagram arrays
acc_threshold	float	0.9	Similarity threshold for pixel-wise accuracy
min_intersection_regions	int	3	Maximum disagreement regions for similarity
intersection_tol	int	5	Boundary tolerance in pixels

Returns: list - Selected diverse phase diagrams

Helper functions

- `tolerant_phase_disagreement(A, B, radius=1)` - Binary map of disagreement between two phase diagrams, tolerant to boundary shifts of `radius` pixels

- `num_connected_regions(map)` - Number of connected regions in a binary map
- `num_holes(map)` - Number of topological holes (persistent homology, requires gudhi)
- `topology_metrics(map)` - Tuple (number of connected regions, number of holes)

utils.py

Utility functions for data processing, active learning, and general operations.

Functions

ensure_tensor

Convert input to PyTorch tensor if not already.

```
ensure_tensor(x, dtype=torch.float32, device="cpu")
```

Parameter	Type	Default	Description
x	array-like	required	Input data (array-like, tensor, or scalar)
dtype	torch.dtype	torch.float32	Desired tensor data type (applied when converting non-tensors)
device	str	"cpu"	Desired device ("cpu" or "cuda")

Returns: torch.Tensor - Input as a PyTorch tensor

ensure_numpy

Convert input to NumPy array if not already.

```
ensure_numpy(x)
```

Parameter	Type	Description
x	tensor or array-like	Input data

Returns: np.ndarray - Input as a NumPy array

brute_sample_new_points

Select new sampling points by evaluating the acquisition function on a set of candidates, with spacing constraints when batch sampling.

```
brute_sample_new_points(model, candidates, sampled_points=None, n_sample=1,  
                        frac_distance_thresh=0.1, epsilon=0.05,
```

```
vanilla_acq=False, distance_acq=True, return_index=False,
acquisition_fn=None)
```

Parameter	Type	Default	Description
model	GP model	required	GP model with acquisition method
candidates	torch.Tensor	required	Candidate points for selection, shape (n, d)
sampled_points	torch.Tensor	None	Previously sampled points, shape (m, d)
n_sample	int	1	Number of points to select
frac_distance_thresh	float	0.1	Minimum distance between selected points, in scaled [0,1] units
epsilon	float	0.05	Small value for numerical stability in acquisition
vanilla_acq	bool	False	Whether to use vanilla acquisition function
distance_acq	bool	True	Whether to include distance-based acquisition component
return_index	bool	False	If True, also return indices of selected points
acquisition_fn	callable	None	Custom acquisition function acquisition_fn(model, x) -> tensor of shape (n,); if None, model.acquisition is used

Returns: torch.Tensor or tuple - Selected points, or (selected_points, selected_indices) if return_index=True

gradient_sample_new_points

Select new sampling points using gradient-based optimization of the acquisition function (BoTorch `optimize_acqf`) with spacing constraints when batch sampling.

```
gradient_sample_new_points(model, sampled_points=None, n_sample=1,
                           frac_distance_thresh=0.1, epsilon=0.05,
                           vanilla_acq=False, distance_acq=True,
                           num_restarts=10, raw_samples=512,
                           inequality_constrain=None, ic_generator=None,
                           acquisition_fn=None)
```

Parameter	Type	Default	Description
model	GP model	required	GP model with acquisition method; the search domain is [min_scale, max_scale]

sampled_points	torch.Tensor	None	Previously sampled points, shape (m, d)
n_sample	int	1	Number of points to select
frac_distance_thresh	float	0.1	Distance (scaled units) below which candidates are penalised when batch sampling
epsilon	float	0.05	Small value for numerical stability in acquisition
vanilla_acq	bool	False	Whether to use vanilla acquisition function
distance_acq	bool	True	Whether to include distance-based acquisition component
num_restarts	int	10	Number of random restarts for optimization
raw_samples	int	512	Number of raw samples for initialization
inequality_constrain	callable	None	Nonlinear inequality constraint passed to optimize_acqf (feasible when ≥ 0)
ic_generator	callable	None	Initial-condition generator for constrained optimization (BoTorch gen_batch_initial_conditions if None)
acquisition_fn	callable	None	Custom acquisition function acquisition_fn(model, x); must support gradients

Returns: torch.Tensor - Selected points, shape (n_sample, d)

get_grid

Generate a regular grid (2D, or N-D when x_min and x_max are lists) for phase diagram evaluation.

```
get_grid(x_min=0, x_max=1, grid_size=100, return_coordinates=False,
        y_min=None, y_max=None, device="cpu")
```

Parameter	Type	Default	Description
x_min	float or list	0	Minimum x-coordinate(s). If list, creates N-D grid.
x_max	float or list	1	Maximum coordinate(s). If list, creates N-D grid.
grid_size	int	100	Number of points along each axis

return_coordinates	bool	False	If True, also return coordinate vectors
y_min	float	None	Minimum y-coordinate (only for 2D, defaults to x_min)
y_max	float	None	Maximum y-coordinate (only for 2D, defaults to x_max)
device	str	"cpu"	Desired device ("cpu" or "cuda")

Returns: torch.Tensor or tuple:

- If return_coordinates=False: grid points, shape (grid_size^N, N)
- If return_coordinates=True, 2D mode: (grid_points, x_coords, y_coords)
- If return_coordinates=True, N-D mode: (grid_points, coord_list), a list of coordinate vectors for each dimension

scaler

Scale data to [0,1] range using min-max normalization.

```
scaler(x, min_scale, max_scale)
```

Parameter	Type	Description
x	torch.Tensor	Input data to scale
min_scale	torch.Tensor	Minimum values for each dimension
max_scale	torch.Tensor	Maximum values for each dimension

Returns: torch.Tensor - Scaled data in [0, 1] range

set_seeds

Set random seeds for reproducibility across PyTorch and NumPy.

```
set_seeds(seed)
```

Parameter	Type	Description
seed	int	Random seed value

visualization.py

Visualization tools for phase diagrams, probability maps, and acquisition functions.

Functions

model_diagram_plot

Generate various 2D plots from a trained GP model.

```
model_diagram_plot(model, plot_type="phase", x_min=None, x_max=None,
                    y_min=None, y_max=None, grid_size=100,
                    sampled_points=None, phase_labels=None, title=None,
                    xlabel="Parameter 1", ylabel="Parameter 2",
                    figsize=(7,6), cmap=None, plot_boundary=False)
```

Parameter	Type	Default	Description
model	GP model	required	Trained GP model with predict/predict_proba/ acquisition methods
plot_type	str	"phase"	Type of plot ('phase', 'probability', 'acquisition')
x_min	float	None	Minimum x-coordinate for plot domain
x_max	float	None	Maximum x-coordinate for plot domain
y_min	float	None	Minimum y-coordinate for plot domain
y_max	float	None	Maximum y-coordinate for plot domain
grid_size	int	100	Number of grid points in each dimension
sampled_points	torch.Tensor	None	Already sampled points (for acquisition plot)
phase_labels	list	None	Names for phase 0 and phase 1 (used by the 'phase' plot)
title	str	None	Plot title
xlabel	str	"Parameter 1"	Label for x-axis
ylabel	str	"Parameter 2"	Label for y-axis
figsize	tuple	(7, 6)	Figure size as (width, height) in inches

cmap	str	None	Colormap name (defaults based on plot_type)
plot_boundary	bool	False	Whether to show phase boundaries as contour lines

Returns: tuple (fig, ax) - Matplotlib figure and axes objects

phase_diagram_plot

Plot a phase diagram with optional contour lines at phase boundaries.

```
phase_diagram_plot(phase_diagram, x_coords=None, y_coords=None,
                    phase_labels=None, title=None, xlabel="Parameter 1",
                    ylabel="Parameter 2", figsize=(7,6),
                    cmap=ListedColormap(["#be96c5ff", "#fdc684ff"]),
                    plot_boundary=False)
```

Parameter	Type	Default	Description
phase_diagram	np.ndarray	required	2D array of phase indices, shape (n_y, n_x)
x_coords	array-like	None	X-axis coordinate values
y_coords	array-like	None	Y-axis coordinate values
phase_labels	list	None	Names for each phase
title	str	None	Plot title
xlabel	str	"Parameter 1"	X-axis label
ylabel	str	"Parameter 2"	Y-axis label
figsize	tuple	(7, 6)	Figure size (width, height) in inches
cmap	str/colormap	two-colour ListedColormap	Colormap for phases (purple for phase 0, orange for phase 1)
plot_boundary	bool	False	Whether to draw black contour lines at boundaries

Returns: tuple (fig, ax) - Matplotlib figure and axes objects

phase_diagram_probability_plot

Plot phase probabilities with decision boundary at p=0.5.

```
phase_diagram_probability_plot(phase_probabilities, x_coords=None,
                              y_coords=None, title=None,
                              xlabel="Parameter 1", ylabel="Parameter 2",
                              figsize=(7,6), cmap='viridis',
                              plot_boundary=True)
```

Parameter	Type	Default	Description
phase_probabilities	np.ndarray	required	2D array of probabilities [0,1], shape (n_y, n_x)
x_coords	array-like	None	X-axis coordinate values
y_coords	array-like	None	Y-axis coordinate values
title	str	None	Plot title
xlabel	str	"Parameter 1"	X-axis label
ylabel	str	"Parameter 2"	Y-axis label
figsize	tuple	(7, 6)	Figure size (width, height) in inches
cmap	str/colormap	'viridis'	Colormap for probability values
plot_boundary	bool	True	Whether to highlight the p=0.5 decision boundary

Returns: tuple (fig, ax) - Matplotlib figure and axes objects

phase_acquisition_plot

Plot acquisition function values for active learning visualization.

```
phase_acquisition_plot(acquisition_values, x_coords=None, y_coords=None,
                       title=None, xlabel="Parameter 1",
                       ylabel="Parameter 2", figsize=(7, 6),
                       cmap='plasma', show_maximum=True)
```

Parameter	Type	Default	Description
acquisition_values	np.ndarray	required	2D array of acquisition values, shape (n_y, n_x)
x_coords	array-like	None	X-axis coordinate values
y_coords	array-like	None	Y-axis coordinate values

title	str	None	Plot title
xlabel	str	"Parameter 1"	X-axis label
ylabel	str	"Parameter 2"	Y-axis label
figsize	tuple	(7, 6)	Figure size (width, height) in inches
cmap	str/colormap	'plasma'	Colormap for acquisition values
show_maximum	bool	True	Whether to mark the maximum acquisition point

Returns: tuple (fig, ax) - Matplotlib figure and axes objects